CADO-NFS on NVIDIA GPUs

Design of the `cado-nfs-gpu` fork

Stefan Teleman `<stefan.teleman@gmail.com>`
in collaboration with Claude Opus 5.5 (Anthropic)

November 09 2026

Abstract

This document describes a fork of CADO-NFS (branch `cado-nfs-gpu`, 68 commits, plus the one adding this document, on top of upstream `master` at `8f0bb2b207`) that runs most of the number field sieve on NVIDIA GPUs: polynomial selection (collision search, size and root optimization), the lattice siever and its cofactorization, the block Wiedemann linear algebra, the character matrix and the square root. Every GPU path gives the same results as the CPU code it replaces: the same polynomials, the same relations, the same matrices and vectors; each was checked against the CPU. The fork also changes the CPU side where the GPU made it the bottleneck (filtering I/O, relation output, the workflow of `cado-nfs.py`) and adds multi-GPU and multi-node operation.

Authorship. The fork was designed, written and measured by Stefan Teleman in collaboration with Claude Opus 5.5, Anthropic's AI model, working in Claude Code: the code, the measurements, the README files and this document were produced together, with every change reviewed, built and tested on the development machine before it was committed.

Hardware. The minimal expected configuration for all of these GPU optimizations is the **NVIDIA RTX PRO 6000 Blackwell**; the optimal configuration is the **NVIDIA B200**. All measurements were made on a development machine with an RTX PRO 4500 Blackwell and an Intel i9-13900K, which is not a production configuration. **Every figure given for the RTX PRO 6000 or the B200 in this document is an estimate; none of them has been tested yet.** Such figures are marked $^{est.}$.

Contents

1 Introduction **3**
1.1 Principles . . . 3
1.2 Hardware . . . 3
1.3 Test numbers . . . 3

2 Building and deployment **4**
2.1 CUDA, CGBN, NCCL and MPI . . . 4
2.2 Target architectures . . . 4
2.3 Running on a GPU without code for it . . . 4
2.4 Choosing a GPU . . . 4

3 Polynomial selection **4**
3.1 Stage 1: the collision search . . . 4
3.2 Size optimization . . . 5
3.3 Root optimization . . . 5

4 Lattice sieving **6**
4.1 Overview . . . 6
4.2 Slabs . . . 6
4.3 Large primes: the walk . . . 6
4.4 The survivors' cofactor checks on the GPU . . . 7
4.5 GPU cofactorization . . . 7
4.6 Top-level bucket fill . . . 7
4.7 Process layout and CPU–GPU overlap . . . 7
4.8 The CPU side of the siever . . . 8
4.9 Designs measured and dropped . . . 8
4.10 Results . . . 8

5 Filtering **8**

6 Linear algebra **9**
6.1 The GPU product . . . 9
6.2 Measured limits . . . 9

7 Characters and square root **10**

8 Several GPUs and several nodes **10**

9 End-to-end results **10**
9.1 c120 . . . 10
9.2 c140 . . . 11
9.3 The production GPUs (estimates, not tested) . . . 11

10 Verification **11**

11 Bugs found and fixed **12**

12 Limitations and future work **12**

A Commits **12**

1 Introduction

CADO-NFS factors integers with the number field sieve in five stages: polynomial selection, lattice sieving, filtering, linear algebra and the square root. On a CPU, sieving dominates (about 60% of a c120 and more for larger numbers), followed by polynomial selection and linear algebra. This fork moves the arithmetic-heavy parts of every stage to NVIDIA GPUs, while keeping the CPU code as the reference and as the fall-back.

1.1 Principles

1. **Exactness.** A GPU path must produce what the CPU path produces, bit for bit: the same polynomials and scores, the same relations (compared after sorting, since threads change their order), the same matrix, the same Krylov vectors, the same square roots. Floating-point code on the GPU repeats the CPU's operations in the same order, compiled without contraction (`-fmad=false`); where a library function differs (CUDA's `log`), the GPU carries an error bound and hands close decisions back to the CPU. Every GPU module has a check mode that compares it with the CPU.
2. **The CPU code stays.** Every GPU path can be turned off by an environment variable, and a binary without code for the GPU it runs on falls back to the CPU (except bwc with `mm_impl=cuda`, which stops with a message).
3. **Designed for the production GPUs.** Choices and defaults are made for the RTX PRO 6000 and the B200, not for the development card. Where these differ, the fork favours the production GPUs, even when the change is slower on the RTX PRO 4500 (Section 4.4).
4. **Measured before kept.** Each change was timed against the previous build, on the same inputs, before it was committed; changes that did not pay were reverted and documented (Section 4.9).

1.2 Hardware

	RTX PRO 4500 (development)	RTX PRO 6000 (minimum)	B200 (optimal)
Architecture	Blackwell, sm_120	Blackwell, sm_120	Blackwell, sm_100
SMs	82	188	148
Memory	32 GB GDDR7	96 GB GDDR7	180 GB HBM3e
L2 cache	64 MB	128 MB	126 MB
FP64 : FP32	1 : 64	1 : 64	about 1 : 2
Role	all measurements	not tested	not tested

The development host is an Intel i9-13900K (8 P-cores and 16 E-cores, 32 threads) with 128 GB of RAM, Fedora 41, CUDA 13.2. Its asymmetric core topology exposed two upstream bugs in `las`'s thread placement (Section 11).

1.3 Test numbers

- **c120** and **c140**: products of two random primes of 60 and 70 digits, factored with `cado-nfs.py -t 32` and the default parameter files.
- **Sieving benchmarks**: a c120 range (special-q 1000000–1030000, $I = 12$) and a c140 range (special-q 2400000–2425000, $I = 13$, 1716 special-q), run with `las -t Machine,16,2`.
- CADO-NFS's own test suite (`ctest`), run for every change on the affected components.

2 Building and deployment

2.1 CUDA, CGBN, NCCL and MPI

CUDA is detected at configure time (`ENABLE_CUDA`, on when `nvcc` is found). Multiple-precision arithmetic on the GPU uses NVIDIA's CGBN (header-only; looked up in `\$CGBN_HOME/include`, then the system include directories). Multi-GPU linear algebra uses NCCL when found (`ENABLE_NCCL`) and MPI when the build is configured with `MPI=1` in the environment. The build options and every runtime switch are described in the companion document *CADO-NFS GPU fork: build options and runtime knobs* (`doc-gpu/cado-nfs-gpu-knobs.pdf`).

2.2 Target architectures

`CMAKE_CUDA_ARCHITECTURES` decides which GPUs the binaries can run on. If it is not set, the fork takes `\$CADO_CUDA_ARCHITECTURES`; else the GPUs of the build host (`native`) when `nvidia-smi` finds one; else `100-real;120-real;120-virtual`, which covers the B200, the RTX PRO 6000 and later GPUs through PTX. The last is the setting for clusters, whose login nodes often have no GPU. All CUDA code builds for sm_100; it has not run on an sm_100 GPU yet.

2.3 Running on a GPU without code for it

A binary built `native` on one kind of GPU has no code for another. `utils/cado_cuda_check.cuh` launches a probe kernel once per module; if it fails, the module says so once and uses its CPU path, with the same results. bwc with `mm_impl=cuda` has no CPU path at that point and stops with a message.

2.4 Choosing a GPU

A node of production GPUs has several (eight on a B200 node), and `cado-nfs.py` runs many GPU programs on it at once. A program that is not told which GPU to use (a `CADO_*_GPU_DEVICE` variable, or an MPI or Slurm local rank) claims one through lock files in `\$TMPDIR/cado-gpu-<uid>` (`utils/gpu_claim.cpp`): the processes take the GPUs in turn, slot 0 of each GPU first, then slot 1, and so on, and a lock disappears with its process. Before this, every `las` used GPU 0. `las` decides once at start (the variable, else the rank, else a claim) and passes the choice to all its GPU parts, so that the siever, the bucket fill and the cofactorization use the same GPU. `CADO_GPU_SIMULATE_NDEV` simulates several GPUs on one for testing.

3 Polynomial selection

3.1 Stage 1: the collision search

Kleinjung's algorithm looks, for each leading coefficient a_d and each special-q root (q, r_q), for two roots r of $(m_0 + x)^d \equiv N \pmod{p^2}$ (for primes p up to P) whose values $u = (r - r_q)/q^2 \bmod p^2$ in $[-M, M)$ are equal. On the CPU a decisional pass with 32-bit keys flags candidate r_q, and an exhaustive pass finds the collisions.

The GPU (`polyselect_gpu_kernels.cu`) takes every (q, r_q) in the CPU's order: it computes $1/q^2 \bmod p^2$ for every (q, p) in 64-bit Montgomery arithmetic, partitions all values u into bins by their high bits (stage A), then buckets each bin by its low bits in shared memory and searches the buckets for equal values (stage B). That test is exact. The flagged r_q go through the unchanged CPU passes in order, so the matches and polynomials are the CPU's. A bin overflow only sets the flag.

Work unit	CPU	GPU
c120, 2 values of a_d, `-t 1`	46.0 s	19.3 s
c140, 1 value, `-t 4`	75.3 s	21.8 s
c160, 1 value, `-t 8`	208.3 s	26.5 s
c180, 1 value, `-t 8`	9.8 s	0.9 s
c220 (degree 6), 1 value, `-t 8`	21.5 s	1.8 s

The check mode (`CADO_POLYSELECT_GPU_CHECK=1`) compares every flag with the exact answer and with the CPU's decisional pass: no flag missed at any size from c100 to c220.

3.2 Size optimization

After the collision search, each raw polynomial is size-optimized: local descents over translations and rotations that minimize the L^2 skew lognorm. With the collision search on the GPU, this is 77–84% of a polyselect work unit, and 450 s of the c140's 530 s of polynomial selection.

An exact GPU version. `sopt_gpu_kernels.cu` runs whole local descents, one per thread. The integers are fixed-width two's complement (a value that does not fit returns the descent to the CPU). The floating-point part is the CPU's operation for operation, built with `-fmad=false` and keeping the CPU's explicit `fma` calls: the conversion of the coefficients, the derivative, the root bound, the real roots by false position, the lognorm. Only `log` differs: CUDA's is within 1 ulp, glibc's within 0.52 ulp, and IEEE-754 does not require either to be correctly rounded. Each value therefore carries an error bound, and a comparison closer than the two bounds stops the GPU: in the main loop the CPU continues from the state at the start of that iteration, in the initial phase it redoes the descent. On 9,648 descents of the c120's pairs the GPU gave the CPU's polynomials and return values (1,134 finished by the CPU).

When it is used. The descents are double-precision code. The RTX PRO 4500 and the RTX PRO 6000 run FP64 at 1/64 of their FP32 rate; there, 16 clients sharing the GPU took 57.9 s against 39.7 s on the CPU for the c120's 22 work units. The GPU path is therefore the default only where the FP32 : FP64 ratio is at most 4 (B200, H100); on the B200 it should be about 30 times faster per $\mathrm{SM}^{est.}$. Two mixed designs (a thread per process sending batches to the GPU, the others on the CPU) were measured and dropped: the batch preparation costs more CPU than the CPU path does for the whole job.

The CPU path. Half of its time is real root finding on double polynomials (Horner evaluations inside false position). The default build has no `-mfma`, so every `std::fma` was a call into libm through the PLT, about 9% of the samples. `cado_math_aux::fma` now issues the FMA instruction inline when the CPU has it (checked once with `__builtin_cpu_supports`); a fused multiply-add is correctly rounded either way, so the output is unchanged. c140 work unit: size optimization 40.2 s $\rightarrow$ 37.6 s.

3.3 Root optimization

Almost all of `polyselect_ropt`'s time is in `ropt_stage2`, called a few hundred times per polynomial by the tuning: a root sieve over an int16 array, then `MurphyE` for the best entries, whose `get_alpha` counts the roots of f modulo every prime up to 2000. On the GPU (`ropt_gpu_kernels.cu`) the sieve lines are recorded by the CPU and applied by the GPU with int32 atomics; since the subtractions commute modulo 2^{16}, the array is the CPU's. The root counts are done by one block per (polynomial, prime); `get_alpha` takes them through a per-thread hook and sums in the same order. One c120 polynomial: 7.9 s $\rightarrow$ 2.6 s; the c120's 48 polynomials:

64 s → 23 s. Three bugs of the CPU sieve were found on the way (Section 11). `cado-nfs.py` now gives root optimization one polynomial per client thread per work unit.

4 Lattice sieving

4.1 Overview

For each special-q, `las` sieves a region of $2^I \times J$ positions (i, j) on both sides, finds the survivors (positions where both norms are nearly smooth), and cofactors them. The GPU siever (`sieve/las-sieve-gpu*.cpp/.cu`) does the whole sieve: lognorms, bucket and small sieving, survivor search and resieving. It is used wherever it applies (two sides with factor bases, no sublattices, $I \leq 16$, resieving on: all default parameter files). The CPU keeps the small sieve set-up per special-q, and per survivor the trial division with the primes the GPU found, the norm checks and the relation output.

Step	GPU method
Lognorms	`lognorm_smart`'s piecewise-linear approximation, row by row (rational side) or per segment and row (algebraic side), the same double operations (`-fmad=false`).
Bucket-sieved primes	The root transform, Franke–Kleinjung reduction and walk of `las-plattice.hpp`, one thread per (prime, root), adding $\log p$ to 16-bit sums; walks continue from one slab to the next.
Small primes	The entries `small_sieve_init` made, applied in shared-memory tiles of 2^{14} positions.
Survivors	`search_survivors`' test, 16 positions per thread.
Resieving	The bucket lattices walked again over a bitmap of the survivors; the small entries tested per survivor.
Cofactor filter	The survivors' cofactor checks in CGBN (Section 4.4).
Cofactorization	P−1, P+1 and ECM chains (Section 4.5).

4.2 Slabs

The region is processed in slabs. A subjob's arrays (norms, 16-bit sums and the survivor array for both sides) take 16 bytes per position, and 16 subjobs share the L2 cache. Slabs were first 2^{25} positions (512 MB per subjob at $I = 14$); at 2^{22} they fit the RTX PRO 4500's 64 MB L2, and $I = 14$ went from 13.09 s to 6.13 s. Each slab also reads and writes every lattice's walk state, so the best slab grows with the factor base: 2^{22} for the c120 (426,000 bucket entries), 2^{23} for the c140 (1.63 million). The default is 2^{22}, plus one power of 2 per doubling of the L2 over 64 MB, plus half a power per doubling of the entries over 426,000, clamped to 2^{17}–2^{26}. The RTX PRO 6000 (128 MB) and B200 (126 MB) get one power more than this card$^{est.}$; `CADO_LAS_GPU_SLAB_LOG` sets the size outright. Relations do not depend on it.

4.3 Large primes: the walk

The bucket-sieved primes are the bulk of the work (at $I = 14$, about 59 million hits per side and special-q, each a global atomic). Several designs were measured; the walk of the Franke–Kleinjung lattice, which visits only the hits, won. Two refinements:

- **Walk state split.** Each lattice's 32-byte state was read and written for every slab, although only the position changes and most lattices have at most a few points per slab. The steps (with $\log p$ in their top bits) are now written once per special-q and the positions kept in two 8-byte arrays swapped after each slab. c140: 6.76 s → 5.84 s.

- **Fused passes.** `lsg_resieve_sieve` resieves slab s and sieves slab $s+1$ in the same pass, so the per-lattice work is done once per slab. c140: 5.71 s → 5.63 s; c120 at $I = 14$: 6.04 s → 5.59 s.

4.4 The survivors' cofactor checks on the GPU

This change was made for the production GPUs. With a GPU 2.3 times or more faster than the development card$^{est.}$, `las` becomes CPU-bound, and the CPU's main work was GMP code on every survivor, 97% of which it then rejected. `las-sieve-gpu-cofilter.cu` now does, for each survivor and side, what `cofactoring_sync` does before cofactorization: the norm $|F(i,j)|$ in CGBN (256 or 512 bits, from the coefficient sizes), the removal of the powers of 2, of the primes of its hits (the hit lists sorted by CUB), of the trial-division primes and of the special-q's factors; then `check_leftover_norm`'s tests (bounds, gaps and a base-2 strong probable prime test). Up to 64 bits the prime test runs in 64-bit Montgomery arithmetic, the same test as `modredcul_sprp2`; checked on 302,019 values against a reference, 0 differences. The kernel went 752 → 155 μs at the c140.

Survivors rejected on a side are dropped on the GPU; the CPU redoes everything for the others, so the relations are unchanged (the survivors entering cofactorization were compared with the filter on and off). On the GPU-bound development card this costs time (full c120: 385 s → 429 s) but saves a quarter of the CPU (3739 → 2797 CPU-seconds), which is what a faster GPU waits on. `CADO_LAS_GPU_COFILTER=0` restores the previous behaviour.

4.5 GPU cofactorization

`facul-gpu` runs the method chains of the cofactorization strategy, one thread per (candidate, side), in 64- or 128-bit Montgomery arithmetic. Stage 1 interprets CADO-NFS's own PRAC bytecode (ECM, P+1) or uses the P−1 exponent; stage 2 runs `stage2.cpp`'s plan with the same backtracking; found factors go through the CPU's tests. The curves are computed over $\mathbb{Q}$ on the host. Candidates of all threads are batched. On 1,268,053 three-large-prime candidates the outcome and primes equal the CPU's; one thread, three large primes: 222 s → 19 s. With the default two large primes cofactorization is a small part of `las`.

4.6 Top-level bucket fill

When the GPU siever does not apply but the top bucket level is 1, the GPU fills the top-level buckets (`las-fill-gpu*`): root transform, lattice reduction and enumeration as the CPU does, counting the hits per bucket in shared memory, then writing the updates in bucket order. c120, 32 threads: 8.5 s → 5.6 s, relations identical.

4.7 Process layout and CPU–GPU overlap

`las` is fastest as one process per GPU with many subjobs: `-t Machine,16,2` is 16 subjobs of 2 threads, each with its own special-q. Separate processes take turns on the GPU (16 × `las -t 2`: 4.9 s), subjobs of one process share it (2.5 s). Within a subjob, the cofactoring tasks of a special-q sieved on the GPU are not waited for: the subjob sieves the next special-q in a second `nfs_work` meanwhile (c120: 2.98 s → 2.45 s). Waits on the GPU sleep on events instead of spinning, which took `las` from about 1700% to 1150% CPU at the same wall time.

`cado-nfs.py` sets this up by itself when it finds GPUs: the las threads are the GPU's share of the node in 2-thread subjobs, at most one sieving work unit per GPU runs at a time, and work units are five times longer (each `las` pays about 0.3 s to start). For nodes that differ, `tasks.sieve.las.threads=gpu-auto` lets each `las` compute its own layout from its CPU affinity mask and the GPUs it sees.

4.8 The CPU side of the siever

With the sieve on the GPU, the CPU's share per special-q is what a faster GPU waits on, especially on nodes with few threads per GPU (a B200 node has about 28). Profile of the c140 benchmark: relation output 23% of the CPU samples, the per-special-q norm set-up 13–17%, the GPU sieve's host side 11–14%, the survivors' trial division 15%. Relation output now formats the 64-bit primes directly instead of building a relation with two `mpz` per prime (byte-identical output; user CPU 16.1 → 15.1 s). The norm set-up iterates its root finding to the last bit, and anything faster would change the norms; it was left as is.

4.9 Designs measured and dropped

- Bucket records (writing every hit, resieving by a scan): the kernels are latency-bound with little work per launch, not bound by the hits.
- One launch for both sides (4 times wider): `las` 3% slower.
- Bucket primes below 2^{15}–2^{18} sieved in the shared-memory tiles: slower at every bound (primes × rows instead of hits).
- Norms per 2^{26} positions, FP32 norms per row, testing survivors against each lattice instead of a second walk, batched resieve walks: slower or no change.
- Other allocators (jemalloc, tcmalloc) for the CPU side: no change.

The measured limit: with 16 subjobs the SMs are active 94% of the time, warp slots 79% full, but only 23% of issue slots used and DRAM at about half its bandwidth: the GPU is full of warps waiting on memory, mostly the large-prime walks' atomics.

4.10 Results

`las` on the RTX PRO 4500	c120 range	c140 range
CPU only, `-t 32`	8.32 s (8000 q)	—
First GPU siever, `-t 32`	3.65 s (8000 q)	—
Final, `Machine,16,2`	2.28 s (30000 q)	5.44 s (25000 q)

The c140 range went 7.62 s → 5.44 s over the last series of kernel changes, with identical relations at every step.

5 Filtering

Filtering is CPU work (reading, parsing and hashing relations), but with the sieve on the GPU it became a visible share of the run, and `cado-nfs.py` repeats it until there are enough relations. Changes, each with identical output:

- **Parallel reading.** `filter_io` handed relations over one at a time under a lock (dup2 got slower with more threads); it now passes batches of 64, and parsing runs in threads of its own (`CADO_FILTER_PARSE_THREADS`). dup2 on a c120 slice: 17.9 s → 8.5 s.
- **Renumbering index.** A coarse index restricts the search for each prime above 2^{20} to one block (15–22% of dup2).
- **Compression.** `.gz` files go through `pigz` when it is installed; files the server's tasks write and reread are zstd (`tasks.compression=auto`): decompressing the c120's dup2 output takes 0.35 s instead of 3.45 s.

- **Fewer rounds.** When purge finds too few relations, `cado-nfs.py` extrapolates from the measured slope instead of asking for the shortfall (c120: 7 rounds → 3), and dup2 processes its slices in parallel.

c120 filtering: 3.3 min → 43.5 s. The GPU keeps sieving during the filtering checks (the server keeps work units queued), so the checks cost no GPU time.

6 Linear algebra

6.1 The GPU product

Block Wiedemann over GF(2) (`bwc.pl`) spends its time in sparse matrix–vector products. The `cuda` matmul backend (`matmul-cuda*.cpp/.cu`) uploads the matrix block in the `matrix_u32` layout, cut into work items of at most 4096 entries (rows are very uneven); one warp per item gathers and XORs the source elements, and the pieces of split rows are combined with atomic XOR. krylov, mksol and secure keep their vectors on the GPU for a whole interval; the host sees them only at checkpoints. With several GPUs, each MPI job has its block on its own GPU, and a product is followed by an XOR reduce-scatter (NCCL has no XOR reduction: grouped send/receive and an XOR kernel) and an all-gather. krylov's V and A files are byte-identical between the CPU and the GPU and across MPI grids (2×2, 3×1, 1×3).

`bwc.pl` uses the GPU by default in a CUDA build: `mm_impl=cuda`, one MPI job per GPU in an MPI build, and the block size $m = n = 64$ becomes $m = n = 128$, since a 128-bit product costs the GPU about what a 64-bit one does (krylov and mksol need half as many products).

c120 matrix (635,213 rows)	CPU, 32 threads	GPU
secure	20.2 s	4.6 s
krylov	152.2 s	11.8 s
lingen	24.5 s	13.1 s
mksol	76.0 s	6.6 s
bwc total	276 s	40.3 s

6.2 Measured limits

The c140 matrix (1,679,568 rows, 210 million entries) takes 3.34–3.47 ms per product: 210 million random 16-byte reads from a 27 MB vector held in L2, about 2 TB/s of L2 traffic. Fewer lanes per row (16, 8 or 4) gave no gain; the source indices are uniform after balancing (the heaviest 8192 columns hold 1.7% of the entries), so there are no hot columns to keep in shared memory. Wider vectors were measured on the c140:

c140 linear algebra	$m = n = 128$ (default)	$m = n = 256$
per product	3.47 ms	5.02 ms
krylov	99.6 s	83.1 s
lingen (32 threads)	41.1 s	78.0 s
mksol	48.3 s	38.3 s
total	220.4 s	231.9 s

lingen stays on the CPU: 6.8 of its 12.3 s on the c120 are a sequential basecase, and the GPU's 1-bit tensor-core MMA peaks at about the GF(2) multiply-add rate that gf2x reaches with PCLMUL on 32 threads, so a GPU lingen would at best match the CPU on this card.

7 Characters and square root

Characters. `linalg/characters_gpu.cu` computes the character matrix: about 50 Legendre symbols of $br - a \bmod p$ per pair (64-bit arithmetic, a binary Jacobi algorithm) and the sign characters, whose sign of $F(a, b)$ is computed exactly in CGBN (256 to 2048 bits). c120: CPU time 16.4 s → 2.9 s, identical output.

Square root. For each dependency, `sqrt` multiplies about a million $(a - b\alpha)$ in product trees and lifts the algebraic square root by Newton's method to tens of millions of bits; GMP's FFT multiplication was most of the time. Large products now go to the GPU (`sqrt_gpu_mul.cu`): an NTT modulo three primes of the form $c \cdot 2^{40} + 1$ above 2^{61}, CRT on the GPU, carries on the host; the product is exact, the same integer as `mpz_mul`'s. A 2^{20}-limb product: 348 ms → 21 ms. These sizes are far beyond CGBN's fixed widths, hence an NTT. `cado-nfs.py` also runs up to four dependencies at once, as memory allows, each on its own GPU when the node has several. c120 square root: 27.9 s → 13.1 s.

8 Several GPUs and several nodes

- **On a node:** GPU claiming (Section 2.4) gives each `las`, polyselect, sqrt and characters process a GPU of its own; sqrt spreads its dependencies over the node's GPUs; bwc runs one MPI job per GPU with NCCL.
- **Sieving across nodes:** `cado-nfs-client.py` sends its host name and its node's GPU count with each request, and the server limits the sieving work units per node to its GPUs (before, the limit was the server's own GPU count for the whole cluster). `CADO_LAS_NGPU` tells a server without GPUs what the sieving nodes have.
- **Linear algebra across nodes:** given several hosts (`hosts=`, `hostfile=` or a Slurm allocation), `bwc.pl` runs nodes × GPUs-per-node MPI jobs, mapped by node, each on the GPU of its local rank.

All of this was tested on the single-GPU development machine only, with simulated GPUs (`CADO_GPU_SIMULATE_NDEV`, `CADO_BWC_SIMULATE_GPUS`) and clients posing as several nodes (`CADO_CLIENT_HOST`, `CADO_CLIENT_NGPU`): the results were correct and the limits as intended. **No real multi-GPU node or multi-node cluster has been tested.**

9 End-to-end results

9.1 c120

`cado-nfs.py -t 32`, c120, RTX PRO 4500	Wall time
Upstream CADO-NFS, CPU only	28.6 min
+ GPU linear algebra	25.4 min
+ GPU siever, cofactorization and polynomial selection	15.1 min
+ filtering changes	13.0 min
+ larger work units, 16 subjobs, kernel tuning	7.2 min (429 s)
+ CPU–GPU overlap in the subjobs	6.7 min (401 s)
+ slabs sized for the L2, sleeping waits	6.4 min (385 s)
+ GPU cofactor filter (slower here, by design; Section 4.4)	7.2 min (429 s)

9.2 c140

c140, `cado-nfs.py -t 32`, RTX PRO 4500	First run	Final run
Polynomial selection	529 s	530 s
Sieving	4674 s	3493 s
Filtering	239 s	245 s
Linear algebra	223 s	221 s
Characters and square root	43 s	48 s
Total	**5725 s**	**4552 s**

In the final run sieving is 77% of the total and GPU-bound.

9.3 The production GPUs (estimates, not tested)

None of the following has been measured. It extrapolates the c140 from the development card, assuming that GPU-bound stages scale with the number of SMs and the memory bandwidth, and that CPU-bound stages do not change.

c140	RTX PRO 4500 (measured)	RTX PRO 6000$^{est.}$	B200$^{est.}$
Polynomial selection	530 s	530 s (CPU-bound)	much less: size optimization on the GPU
Sieving	3493 s	1500–1750 s	less than the RTX PRO 6000
Filtering	245 s	245 s	245 s
Linear algebra	221 s	130–150 s	less (HBM bandwidth)
Characters, square root	48 s	about 40 s	about 40 s
Total	**4552 s**	**2450–2700 s**	—

On the RTX PRO 6000 polynomial selection and filtering, both CPU-bound, would become about 30% of the run$^{est.}$. The B200's fast FP64 enables the GPU size optimization (Section 3.2). These numbers are to be replaced by measurements as soon as one of these GPUs is available.

10 Verification

- **Check modes**, one per GPU module, compare it with the CPU while running: `CADO_LAS_GPU_SIEVE_CHECK` (every sieve array, 0 mismatches at $I = 12$–16), `CADO_POLYSELECT_GPU_CHECK`, `CADO_ROPT_GPU_CHECK`, `CADO_CHARACTERS_GPU_CHECK`, `CADO_SQRT_GPU_CHECK` (every product).
- **Test programs**: `facul_gpu_test` (cofactorization against `facul_all`), `sopt_gpu_test` (size optimization), `sqrt_gpu_mul_test` (400 products up to 200,000 limbs).
- **Relations** sorted and compared with the previous build after every siever change; **survivors** entering cofactorization compared with the cofactor filter on and off; **bwc files** compared across backends and MPI grids.
- **ctest**: the affected components' tests for every commit (up to 217 at once), and complete factorizations (`full_p30`, `full_c40`, c60–c140) with the factors checked.

11 Bugs found and fixed

- **las thread placement on asymmetric CPUs** (upstream): on a hybrid CPU (P- and E-cores) the topology code divided by zero and `-t auto` was rejected, which made the descent hang; binding specifiers were refused.
- **polyselect_ropt** (upstream, three): an array shorter than one L1 block was not sieved at all; with several rows, the last block of each row but the first was never sieved for simple roots; the tuning size depended on the core the program ran on (64 or 32 KB L1 on a 13900K). Fixing the first improved 3 of the c120's 24 work units.
- **ringbuf::strchr** (upstream): wrong positions past the wrap point.
- **las**: `sublat_bound` tests that could never be true.
- **The c40 QS test hang** (upstream parameters): bwc needs more than $m + n$ rows; `params.qs.c40` kept 32 excess relations, and the matrix sometimes had 127 rows. Purge now keeps at least 160.
- **GPU choice and switches** (in this fork, found while writing the companion document): on multi-GPU hosts `las` used an older lock of its own instead of the shared claim, and its parts read the MPI ranks differently; `cado-nfs.py` and `las` parsed `CADO_LAS_GPU_SIEVE` differently; dup2's thread default never applied.
- **The (1,0) carry** (in this fork's GPU siever, found by the check mode): the 16-bit sum of (0,0) carried into (1,0) for $I \geq 14$.

12 Limitations and future work

- Measure everything on an RTX PRO 6000 and a B200; replace the estimates in this document, and retune the defaults chosen for those GPUs (slab size, subjob layout, the cofactor filter, the GPU size optimization's FP64 threshold).
- Test real multi-GPU nodes and multi-node clusters.
- On GPUs with slow FP64, polynomial selection's size optimization and filtering stay on the CPU and become a larger share of the run.
- The siever's per-special-q norm set-up stays on the CPU, since a faster root finding would change the norms.
- lingen stays on the CPU.

A Commits

The fork's 68 commits, oldest first, and then the commit that adds this document and its companion. Each is also available as a numbered patch (`git format-patch`).

#	Commit	Subject
1	`f84d4731e`	bwc: GPU matmul backend for GF(2) (mm_impl=cuda)
2	`7fb51f732`	bwc: keep krylov/mksol vectors on the GPU; split long rows
3	`affa766e8`	bwc secure: keep my and dvec on the GPU between check stops
4	`be5b88b64`	bwc.pl: use the GPU by default in CUDA builds (gpu=auto|1|0)
5	`1439425d1`	bwc: multi-GPU products with the vectors on the GPUs (NCCL or MPI)
6	`b0ccbe661`	bwc: README.gpu.md (build, defaults, switches, design, c120 results)
7	`c4e69550d`	facul: GPU cofactorization engine (P-1, P+1, ECM chains on the GPU)

#	Commit	Subject
8	357a29b2e	las: cofactorization on the GPU, batched over all threads
9	115d9adde	las: top-level bucket fill on the GPU
10	7545374e0	las: GPU siever, step 1: the sieve arrays (checked against the CPU)
11	1911405c9	las: GPU siever, step 2: survivors and resieving on the GPU, used by las
12	fcf03222f	las GPU siever: faster kernels, tables kept on the GPU
13	8c4ff86ba	las GPU siever: slabs of 2^25 positions, for I up to 16
14	1b948500e	GPU siever on by default; cado-nfs.py runs one las per GPU
15	56b2dbee8	las: sublat_bound tests that could never be true
16	3b96a3403	ringbuf::strchr: wrong positions past the wrap point and past the data
17	4db6484fc	filter_io: relations handed over in batches, parsed by threads of their own
18	90e888db0	renumber: coarse index for the primes above the cache; dup2 -t
19	b48037793	compressed files: .gz through pigz when it is installed
20	b99fd0501	tests: builddep tests of the same target do not run at the same time
21	9cafa7528	cado-nfs.py: fewer filtering rounds, dup2 slices in parallel
22	80170101e	cado-nfs.py: dup2 gets -t
23	50e9ed8ad	polyselect: collision search on the GPU
24	722d062f8	cado-nfs.py: root optimization, one polynomial per client thread
25	e428382ab	polyselect_ropt: root sieve and alpha root counts on the GPU; three sieve bugs
26	eff03594b	polyselect: exact GPU size optimization (L2_skew_lognorm, local descents), not used yet
27	5f2bb068b	polyselect: size optimization descents on the GPU when it has fast FP64
28	ff7edd645	sqrt: large products on the GPU (NTT); several dependencies at once
29	ef1c948d3	bwc.pl: keep m=n=64 on small matrices with the GPU
30	e4429a737	cado-nfs.py: the dependency-file step of sqrt runs with -t 1
31	e79ebd574	characters: the character matrix on the GPU, signs with CGBN
32	451c6b951	tests: full_p30 checksum after the polyselect_ropt sieve fix
33	cdec177f6	README: postprocessing on the GPU, c120 results
34	f0515296e	las: no binding on asymmetric topologies, as intended
35	67a3d869a	cado-nfs.py: larger sieving workunits with the GPU siever
36	2e97a9580	las GPU siever: faster small resieve, small sieve tiles and lognorms; (1,0) carry bug
37	98d474a89	las GPU siever: resieve without waiting for the survivor count
38	2d056fc01	las GPU siever: 16 subjobs of 4 threads per GPU; flat region index
39	70e1cdbbd	las GPU siever: overlap a subjob's cofactoring with the next GPU sieve
40	8a8e5c488	las-parallel: accept binding specifiers on asymmetric topologies
41	0d8a480ee	las GPU siever: slabs of 2^22 positions; waits sleep instead of spin
42	257492730	Docs: GPU lingen and mid-prime tile sieving measured, not adopted
43	f5443093b	las GPU siever: the survivors' cofactor checks on the GPU with CGBN
44	4cf95f2a2	las GPU siever: keep only the hits at survivors that are kept
45	cf21ab9ba	las GPU siever: send back only the kept survivors and their hits
46	92a36fd43	GPU builds for the production GPUs (B200 sm_100, RTX PRO 6000 sm_120)
47	ed9ddeccb	GPU programs claim a GPU each on multi-GPU nodes
48	5c96a1d07	sqrt: the dependencies' GPU products on all the node's GPUs
49	b1f359553	las GPU siever: slab size from the GPU's L2 size
50	bed5f4075	cado-nfs.py: filtering files in zstd when zstd is installed
51	0f0e9e95d	cado-nfs.py: keep at least 160 excess relations for bwc over GF(2)
52	e1a8b2a5b	polyselect README: CPU and GPU together in the size optimization, measured
53	31d7a0bc5	las GPU siever: algebraic norms by segment; true cofactor-filter count
54	20c773764	las GPU siever: slab size also from the size of the factor base

#	Commit	Subject
55	`f12a18afa`	las GPU siever: walk state split into positions and steps
56	`f3cf69a88`	las GPU siever: logp with the walk steps, not read from the lattice
57	`935755cc3`	polyselect README: the GPU helper thread for size optimization, measured
58	`6e2d458cd`	las GPU siever: resieve a slab and sieve the next one in one pass
59	`033322fbc`	las GPU cofactor filter: base-2 SPRP in 64-bit arithmetic up to 64 bits
60	`9d6b609ff`	las GPU siever: survivor search 16 positions a thread; fixed grid for small resieving
61	`f4a9fe553`	cado-nfs.py: sieving workunits limited per node, not cluster-wide
62	`17e365136`	bwc.pl: one MPI job per GPU of every node when given several nodes
63	`b7366d358`	las -t gpu-auto: each node picks its own GPU thread layout
64	`a90da6eb5`	las: write relations directly, without an mpz per prime
65	`54dd6399a`	linalg/bwc/README.gpu.md: the product kernel's limits on the c140
66	`f89c3dde3`	linalg/bwc/README.gpu.md: m=n=256 measured on the c140 (231.9 s vs 220.4 s)
67	`fd2a4ce97`	cado_math_aux::fma: inline FMA instruction instead of a libm call
68	`e80379d4b`	GPU choice, CADO_LAS_GPU_SIEVE and dup2 -t made consistent; docs fixed